

C5 - MULTIMODAL RECOGNITION

Task 3: Image Captioning (I)

Team 1:

Diego Hernández Antón · Oriol Juan Sabater · Valentin Micu Hontan
Xavier Pacheco Bach · Benet Ramió Comas



MASTER IN
COMPUTER VISION
Barcelona



Setup & Introduction

Image Captioning

Process of generating **descriptive textual** sentences that convey the content of an image.



A person riding a motorcycle on a dirt road.

It integrates **Computer Vision (CV)** to automatically analyze visual data and **Natural Language Processing (NLP)** to produce coherent, contextually relevant captions that reflect the objects, actions, and scenes depicted in the images.

Goals

- **Primary:** Establish an Image Captioning baseline using an **Encoder-Decoder architecture** on the **VizWiz dataset**, evaluating performance with BLEU, ROUGE-L, and METEOR.
- **Modifications:** Test architectural changes by swapping the **Encoder** (e.g., ResNet, VGG), **Decoder** (e.g., LSTM), and **Text Representation** (subword, word-level, character).
- **Optional:** Implement an **attention mechanism**.

Frameworks & Models

- **Hugging Face** ([Evaluate Doc](#)):
 - ROUGE-L, BLUE, METEOR
- **OpenAI** ([CLIP](#))
- **Torchvision** ([Models Doc](#))



Hugging Face

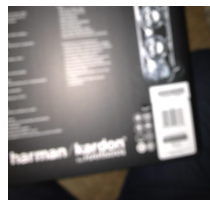
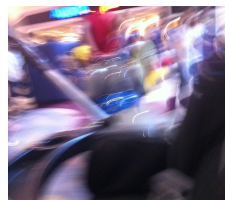


OpenAI

- *We first outline our experimental framework before detailing task results.*
- **Github Repository:** <https://github.com/valy3124/C5-Team1>

Dataset: VizWiz

Note: These images were taken by blind persons. Because the photographers couldn't see the viewfinder, the images often suffer from poor lighting, extreme blur, "finger-over-the-lens," or the subject being entirely out of frame.



3

Model Development Phase

Train

Used for the initial training phase (**90% of the full training data**)

Training images: 20,579
Training captions: 90,432

Dev

Used for hyperparameter tuning and model selection (**10% of the full training data**)

Training images: 2,287
Training captions: 10,096

Final Training and Evaluation Phase

Full Train

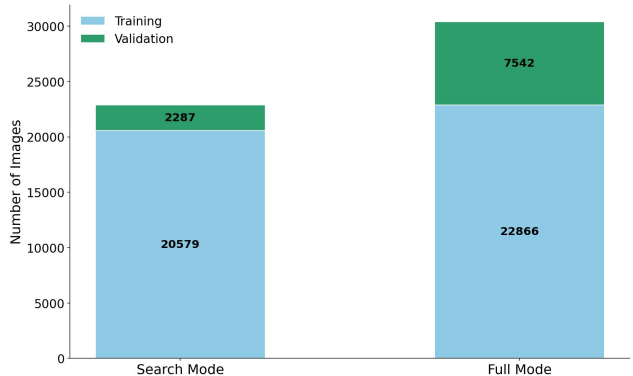
Used for the **final training stage**, combining Train and Train-Dev once the best model and hyperparameters are selected, to maximize available training data.

Testing

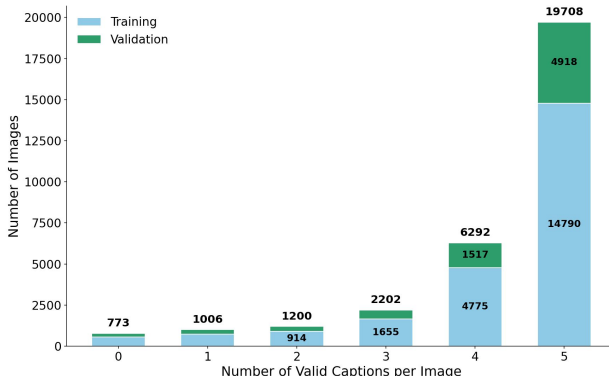
Reserved strictly for final evaluation. We used the official [validation set](#).

Test images: 7,542
Test captions: 33,186

Dataset Distribution: Search vs. Full Mode



Distribution: Captions per Image(Full Data Split)



Key Observation

- There are some images with **no caption** that had to be removed.
- As per project requirement, we do **not use** the dataset's **test set**.
- There is a significant **imbalance** in the number of **captions per image**, which may affect model training.

Metrics Description

We use **METEOR** as our main metric during evaluation because it focuses on **recall** and handles **synonyms** (using a lexical database). This means it checks if the model actually gets the **meaning** of the image right, rather than just demanding exact word matches like BLEU does. We save **CIDEr** strictly for the final comparison between the baseline and our final model. Since CIDEr uses **TF-IDF** to give more weight to specific objects and less to common filler words, it's the most direct way to show whether our architecture changes actually **helped the model** recognize the important visual details in the scene.

*

METEOR

Def: Harmonic mean of precision and recall (favoring recall), with a penalty for fragmentation.

$$F_{mean} \cdot (1 - \text{Pen})$$

where

Why: Rewards capturing the actual meaning and heavily penalizes scrambled word order.

$$F_{mean} = \frac{P \cdot R}{\alpha P + (1 - \alpha) R},$$

$$\text{Pen} = \gamma \left(\frac{c}{m} \right)^\beta$$

BLEU

Def: Measures exact n-gram overlap with references (1 for words, 2 for pairs).

BLEU 1

$$\min \left(1, \frac{\text{output-length}}{\text{reference-length}} \right) \text{precision}_1$$

Why: Verifies exact vocabulary match and penalizes artificially short sentences.

BLEU 2

$$\min \left(1, \frac{\text{output-length}}{\text{reference-length}} \right) \left(\prod_{i=1}^2 \text{precision}_i \right)^{\frac{1}{2}}$$

CIDEr

Def: Measures consensus with human references by weighting specific, informative words using TF-IDF.

$$\frac{1}{m} \sum_{j=1}^m \frac{\vec{g}^n(c_i) \cdot \vec{g}^n(s_{ij})}{\|\vec{g}^n(c_i)\| \|\vec{g}^n(s_{ij})\|}$$

where

Why: Ensures the model identifies key visual objects instead of just repeating common filler words.

- m : No. of human references.
- c_i : Candidate caption.
- s_{ij} : Reference caption j .
- $\vec{g}^n$: TF-IDF weights.

ROUGE-L

Def: Measures the longest common subsequence (LCS) between the generated and reference text.

$$2 \cdot P \cdot RP + R$$

where

$$P = \frac{LCS}{c}, R = \frac{LCS}{r}$$

Why: Ensures the correct order of visual events, even if words aren't perfectly consecutive.

LCS: Length of the Longest Common Subsequence.
c, r: Total words in candidate (c) and reference (r).

*

Approach: Experimental Strategy

Core Evaluation Premises

- **Primary Metric: METEOR** drives all decisions. All reported scores correspond to the model's best METEOR epoch.
- **Data Integrity:** Steps 0–5 use strictly the **Train/Dev** split to prevent data leakage.

Train only the **Baseline** and the **Final Model** on the **Full Training Set**. Evaluate on the **Test Set** ([Slide 3](#)) to compare generalization and perform qualitative analysis.

Step 0: Baseline

- Train **initial architecture** (**ResNet-18 + GRU + Char-level**).
- Set baseline metrics and establish data evaluation methods.

Step 3: Text representation

- **Fixed:** Best Encoder & Decoders.
- **Action:** Evaluate tokenization levels (**Character, Subword, Word**). Select the best performer.

Step 4: Attention Mechanisms

- **Fixed:** Best Encoder, Decoder, & Word-level tokenization
- **Action:** Explore attention types (**Soft, Adaptive, Visual Prefix**) to improve visual grounding.

Step 1: Encoder Search

- **Fixed:** Decoder & Char-level tokenization.
- **Action:** Explore various vision encoders and select the best.

Step 2: Decoder Search

- **Fixed:** Best Encoder & Char-level tokenization.
- **Action:** Explore decoder architectures (**GRU, LSTM, xLSTM**), layer depths, and embedding dimensions. Select the best configuration per type.

Step 5: Hyperparameter search

- **Fixed:** Best overall architecture
- **Action:** Perform a Bayesian sweep (**Optimizer, LR, Scheduler, Decoding method**) to find the optimal final model

STEP 0: Baseline

Setup

- **Baseline Model:** ResNet18 + GRU + Characters
Optimizer: Adam | LR: 5e-5
Scheduler: Plateau | Batch size: 128
- **Data Strategies Tested:**
 - **Random Caption:** Selects one random caption per image per epoch for both training and validation.
 - **Replicated Images (All Captions):**
 - **Duplicates images** so each available caption forms a unique training pair, effectively **multiplying our training samples**.
 - During evaluation, the model's prediction is compared against **all ground-truth references** for that image, returning the **highest matching score**.

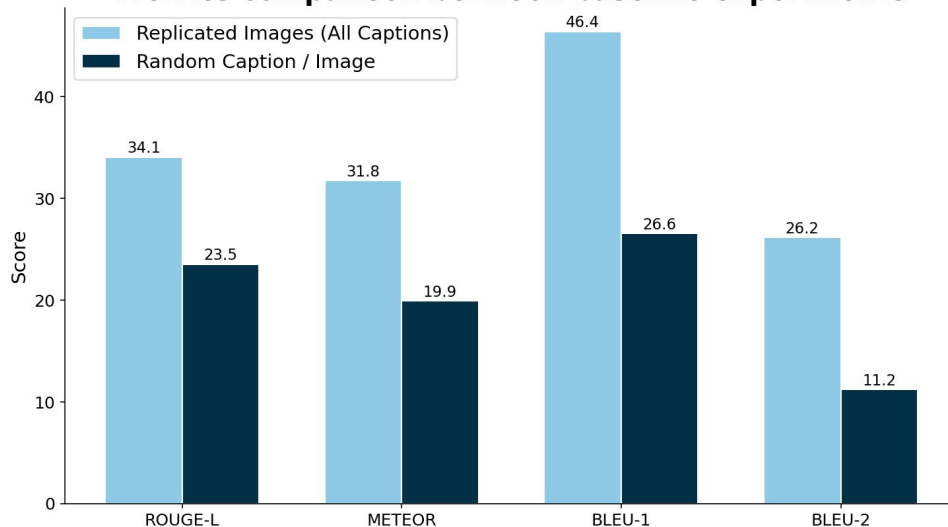
Decision

- **Final Setup:** We will use the **Replicated Images** strategy with multi-reference evaluation for all subsequent experiments in this project.

Results

- The "Replicated Images" strategy **significantly outperforms** the "Random Caption" method across all metrics (e.g., METEOR jumps from 19.9 to 31.8).
- **Why the improvement?** Training on all available captions exposes the model to a **larger volume of data** and more **diverse sentence structures** for the same visual features. Furthermore, evaluating against all references is a **much fairer reflection of performance**, as a single image can have multiple perfectly valid descriptions.

Metrics comparison between baseline experiments



STEP 1: Encoder Search

Setup

- **Baseline:** **Resnet18** (*METEOR*: 31.8)
- **Setup:** Encoder freezed for all the experiments.
- **Encoder sweep:**
 - **CLIP-ViT-b32** (89.5M params)
 - **ResNet-34** (22.9M params)
 - **VGG-16** (16.4M params)
 - **Efficientnet-B0** (6.3M params)
 - **ResNet-50** (26.2M params)
 - **VGG-19** (21.7M params)
 - **CLIP-ViT-L14** (305M params)

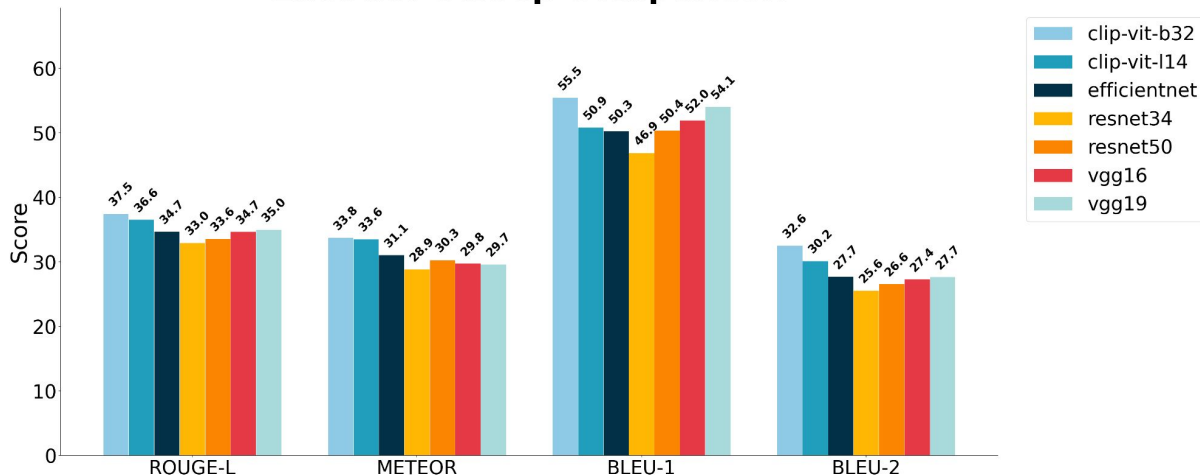
Decision

- **Final model:** CLIP-ViT-b32
- **METEOR:** **33.8**
- **Improvement** **+2.0**

Results

- **CLIP-ViT-b32** achieves the highest scores across all metrics, outperforming all other encoder backbones.
- CLIP is pre-trained on **image-text pairs**, so its features are already **semantically aligned**, making them more useful for the captioning decoder than CNN/EfficientNet features.
- **Model size $\neq$ performance** Larger encoders don't consistently improve results; representation quality matters more than parameter count.

Encoder Sweep Comparison



STEP 2: Decoder Search

Setup

- **Baseline:** GRU (1 layer, 512 dim) (METEOR: 33.8)
- **Setup:** Encoder frozen; search best hidden dim and number of layers per decoder type.
- **Decoder sweep**
 - Types: GRU, LSTM, xLSTM
 - Hidden dim: 512, 1024
 - Layers: 1, 2, 3

Results

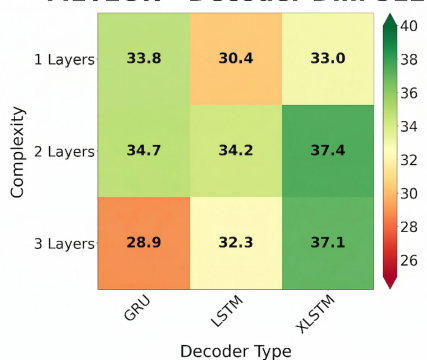
- **Best decoder:** xLSTM consistently achieves the top performance across configurations, with a small but stable margin over LSTM.
- **Depth vs stability:** Performance peaks at **2 layers**, with deeper GRU models degrading sharply, whereas xLSTM maintains stable performance even at higher depth.
- **Width vs architecture:** Increasing hidden size benefits LSTM, but **cannot compensate for weaker architectures** (GRU). Overall, decoder type has a larger impact than dimension size

Decision

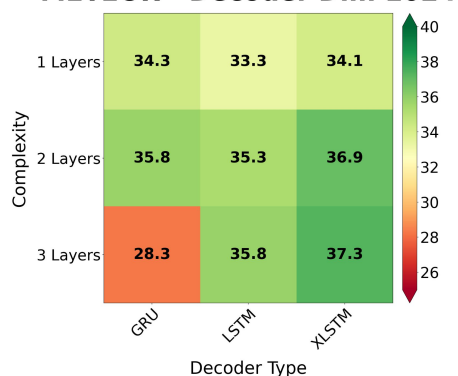
Best configuration per decoder

- **GRU:** 2 layers, 1024 dim
 - METEOR: **35.8 (+2.0)**
- **LSTM:** 3 layers, 1024 dim
 - METEOR: **35.8 (+2.0)**
- **xLSTM:** 2 layers, 512 dim
 - METEOR: **37.4 (+3.6)**

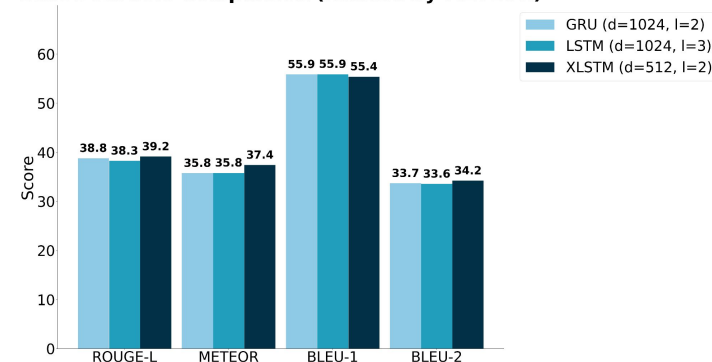
METEOR - Decoder Dim 512



METEOR - Decoder Dim 1024



Best Decoders Comparison (Ranked by METEOR)



STEP 3: Text Representation Search

Setup

- **Baseline** (character tokenization):
 - **GRU**: METEOR **35.8**
 - **LSTM**: METEOR **35.8**
 - **xLSTM**: METEOR **37.4**
- **Setup**: Compare tokenization strategies across decoders
- **Tokenization sweep**: **Character**, **Subword** and **Word**

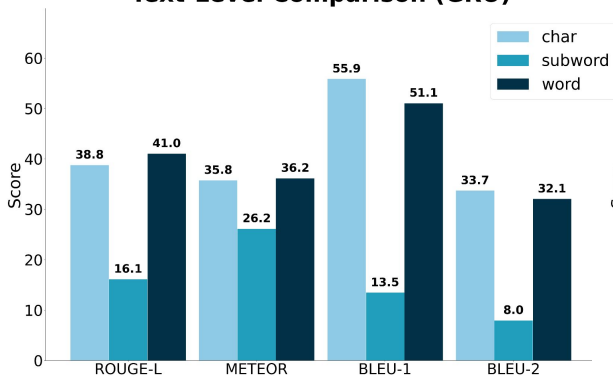
Results

- **Subword collapse**: Subword tokenization performs significantly worse, as models struggle to **properly terminate sequences**, often looping and repeating fragments instead of forming complete words.
- **Word vs character**: Both perform strongly. Word-level provides **direct semantic tokens**, while character-level uses a **closed vocabulary**, avoiding “out-of-vocabulary” issues and enabling flexible word construction.
- **Consistent trend**: The same ranking holds across **GRU**, **LSTM**, and **xLSTM**. Tokenization choice has a larger impact than decoder architecture.

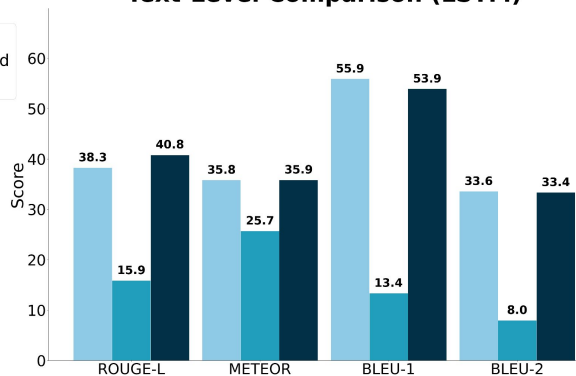
Decision

- We select **word-level tokenization** for attention across all decoders (better semantic alignment)
 - GRU: **METEOR**: 36.2 (+0.4)
 - LSTM: **METEOR**: 35.9 (+0.1)
 - xLSTM: **METEOR**: 37.4 (+0.0)

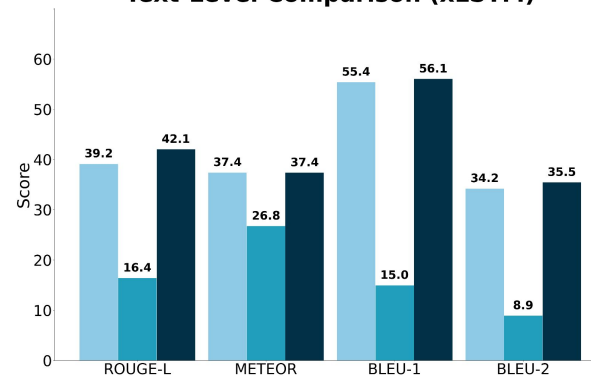
Text-Level Comparison (GRU)



Text-Level Comparison (LSTM)



Text-Level Comparison (xLSTM)



STEP 3: Qualitative text-representation

VizWiz_train_00011300.jpg



We chose a validation set sample to demonstrate the qualitative differences in different text-representation levels using a LSTM decoder.

LSTM - Char level

A black electronic device with a silver and black button and a silver background and a silver background and a silver background with a blue background and white text on it.

LSTM - Subword level

a digital clock on a table with a digital clock .
0 0 . 8 on it . 2 0 1 2 . 1 1 . 0 0 degrees . 0 1 . 0 8 0
/ 2 0 1 2 . 1 2 degrees . 1 3 . 8 degrees
fahrenheit . 2 0 1 2 . 8 0 / 2 2 . 8 0 / 2 2 pm

LSTM - Word level

a digital clock with the time <UNK> and the
time <UNK>



Observations

- **Char-level repetition:** It completely misses the main object (the clock) and gets stuck in a loop repeating colors and backgrounds.
- **Subword-level hallucination:** We see that it correctly identifies the "digital clock," but it wildly invents random dates, temperatures, and numbers to make up for the blurry text.
- **Word-level handles uncertainty with <UNK>:** It provides the most logical sentence structure but outputs <UNK> (Unknown) tokens when it cannot read the specific out-of-vocabulary numbers on the display.

GT Captions

- An analog clock is captured on a wooden desk.
- A cable box for the TV showing the time.
- A gadget with a timer on the kitchen counter top
- A radio is on the bedside table showing time.

**Word and Char-level representations perform better than Subword-level but remain imperfect. Next, we will see new modules that solves these remaining issues.*

STEP 4: Soft Attention (*)

Note: This attention mechanism used in **LSTM** and **GRU**

Motivation: Traditional image captioning compresses the entire image into a single global vector, losing spatial details. **Soft Attention** solves this by computing a differentiable, weighted distribution over all image patches. Unlike **Hard Attention**, which rigidly selects only one single patch at a time and requires complex reinforcement learning to train. Soft Attention smoothly focuses its "gaze" on relevant regions while remaining fully trainable via standard backpropagation.

Step 1: Extract Annotation Vectors

- Pass the image through an encoder to **extract L spatial patches**. Each patch is a vector representing a specific region of the image.
- This will be our **annotations vectors** (one for each patch):

$$a = \{a_1, \dots, a_L\}$$

Step 2: Calculate Attention Weights (Bahdanau Additive Attention)

- At time step t , the decoder's previous hidden state h_{t-1} **evaluates each patch a_i to calculate an alignment score**, which is then converted into a probability distribution (**weights**) that sums to 1.

$$e_{i,t} = v_{att}^T \tanh(W_a a_i + W_h h_{t-1})$$

$$\alpha_{i,t} = \frac{\exp(e_{i,t})}{\sum_{j=1}^L \exp(e_{j,t})}$$

Step 3: Compute the Context Vector

- Multiply each image patch by its corresponding attention weight and sum the results.** This creates a single dynamic vector **focused only on the most relevant visual features for the current word**.

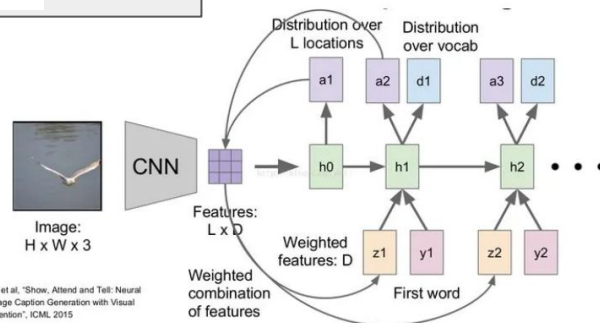
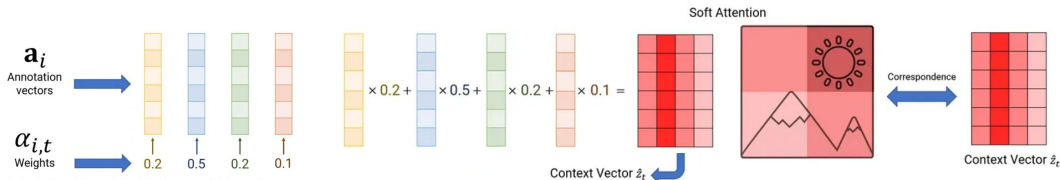
$$\hat{z}_t = \sum_{i=1}^L \alpha_{i,t} a_i$$

Step 4: Update Decoder and Generate

- Feed this visual context vector alongside the previous word's embedding into the decoder cell to **update its memory state and predict the next word**.

$$h_t = \text{Decoder}([E_{y_{t-1}}; \hat{z}_t], h_{t-1})$$

Example for 4 Patches



*

K. Xu, J. Ba, R. Kiros, K. Cho, A. Courville, R. Salakhutdinov, R. Zemel, and Y. Bengio, "Show, attend and tell: Neural image caption generation with visual attention," in *International Conference on Machine Learning (ICML)*, 2015, pp. 2048-2057.

STEP 4: Adaptive Attention (*)

Note: This attention mechanism used in **LSTM** and **GRU**

Motivation: Standard attention mechanisms force the decoder to look at the image for *every* generated word. However, non-visual words (like "the" or "of") and **highly predictable context do not require visual grounding**. **Adaptive Attention** introduces a **Visual Sentinel**, allowing the model to dynamically decide at each time step whether to attend to the image or rely solely on its internal language memory.

Step 1: Generating Visual Sentinel (s_t)

- The sentinel represents the **linguistic knowledge** of the decoder. It acts as a buffer that stores what the model has already learned from the text sequence so far.
- A **gating mechanism** (g_t) decides **how much** of the current RNN state should be saved as a visual fallback.

$$g_t = \sigma(W_g x_t + W_g h_{t-1})$$

$$s_t = g_t \odot f(\text{internal_state}_t)$$

Step 2: Adaptive Attention Scores

- We calculate how well the current state matches the image regions **versus** how much it matches the sentinel.
- Image Scores $z_t(i)$:** Measures relevance of visual region i .
- Sentinel Score $z_t(s)$:** Measures the desire to ignore the image and rely on the language model.

$$z_{t,i} = w^T \tanh(W_s s_t + W_g h_t)$$

$$z_{t,s} = w^T \tanh(W_s v_i + W_g h_t)$$

Step 3: The Sentinel Gate (β_t)

- This is the Decision Step. We use a Softmax to create a probability distribution **over the image AND the sentinel**.
- The **last element $k+1$ is the Sentinel Gate β_t :** the specific weight given to the sentinel.

$$\hat{\alpha}_t = \text{softmax}([z_t; z_{t,s}]) \quad \hat{\alpha}_t = [\underbrace{\alpha_{t,1}, \alpha_{t,2}, \dots, \alpha_{t,k}}_{\text{Spatial Attention (a)}} \quad \underbrace{\beta_t}_{\text{Sentinel Gate}}]$$

Step 4: Building the Adaptive Context Vector and Word prediction

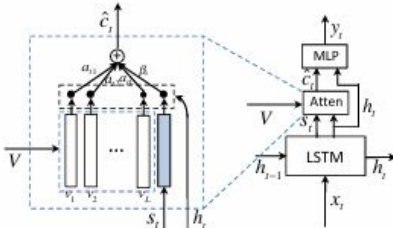
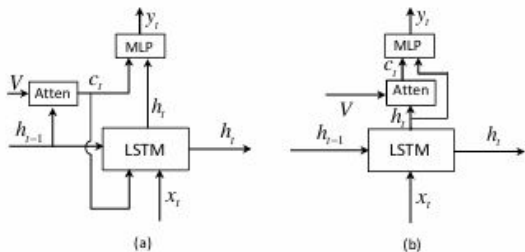
- The final context vector is a **mixture of the visual features and the sentinel**, regulated by the weight β_t .
- The model looks at **both the Adaptive Context and its own Hidden State** to pick the next word.

$$\hat{c}_t = \beta_t s_t + (1 - \beta_t) c_t$$

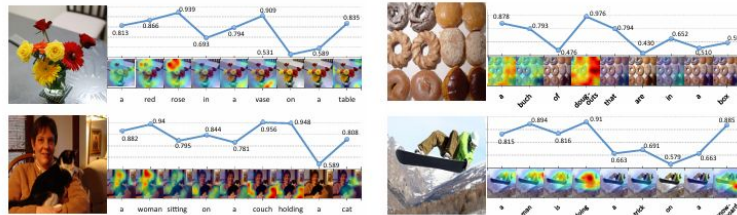
$$P(y_t) = \text{softmax}(W_p(\hat{c}_t + h_t))$$

Soft Attention (a) vs Adaptive Attention (b)

Adaptive Attention (more detail)



Visual Grounding probabilities of each generated word and corresponding spatial attention maps



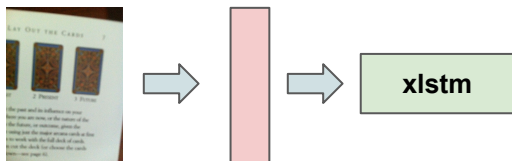
* J. Lu, C. Xiong, D. Parikh, and R. Socher, "Knowing when to look: Adaptive attention via a visual sentinel for image captioning," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 3242-3250.

STEP 4: "Visual Prefix" in xlstm

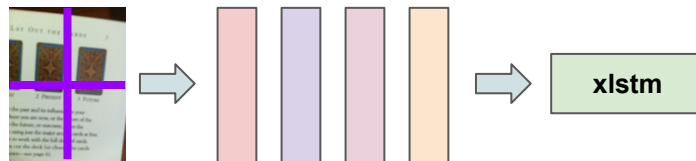
Note: Only used in **xLSTM**. This is **not an attention mechanism** (it adds unnecessary complexity)

Motivation: Traditional models suffer from a spatial information bottleneck by compressing the entire image into a single vector. To avoid the high computational cost of step-by-step explicit attention, here we use a **Visual Prefix**. By sequentially injecting image patches before the text, the xLSTM's novel **Matrix Memory** natively absorbs and retains complex spatial relationships.

Default Strategy



Visual Prefix



Explanation

- **Sequential Visual Prompting:** Instead of applying global pooling, the uncompressed spatial feature map is flattened into a sequence of N patches. These are concatenated with a learnable **<SEP>** token to form a unified multimodal input stream: **[Patch₁, ..., Patch_N, <SEP>, Word₁, ...]**.
- **Covariance Matrix Conditioning:** As the xLSTM sequentially processes the visual prefix, the mLSTM's internal covariance memory matrix C_t iteratively absorbs the visual patches. This builds a rich, structural representation of the image space within the recurrent state.
- **O(T) Inference Efficiency:** The decoder auto-regressively predicts words **by querying this heavily conditioned matrix memory**. This allows the model to ground words to spatial regions **without computing expensive** explicit attention weights at every step, reducing complexity to $O(T)$.

STEP 4: Attention Quantitative

Setup

- We explored **two attention mechanisms** for **GRU** and **LSTM** decoders: **Soft Attention** and **Adaptive Attention**.
- For **xLSTM**, we adopted a **visual prefix** approach, representing the image as a sequence of visual tokens instead of using explicit attention.

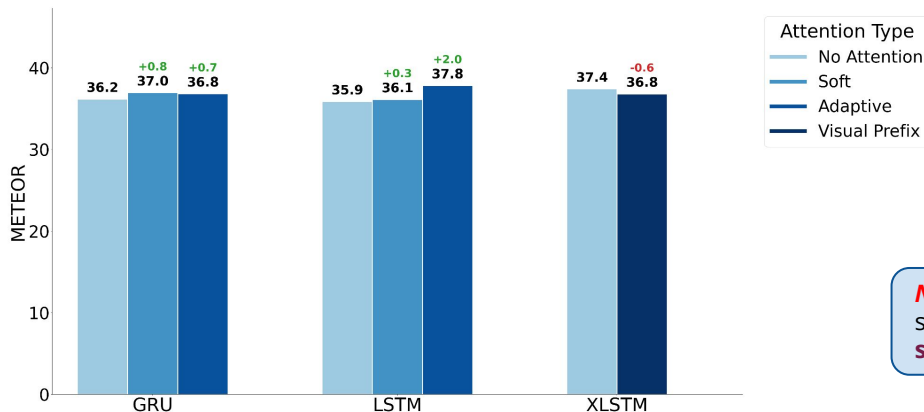
Results

- Attention **consistently improves performance over the baseline** for **GRU** and **LSTM**. For **GRU**, both attention mechanisms yield **similar gains**, while for **LSTM**, **Adaptive Attention** significantly **outperforms Soft Attention**, likely due to its ability to **dynamically balance visual information and language context via the sentinel mechanism**.
- For **xLSTM**, the **visual prefix does not improve performance**. This is likely because **xLSTM** already integrates long-range dependencies through its internal memory, and **lacks explicit mechanisms to re-attend to spatial features**, making it harder to refine visual grounding during generation.

Decision

- **Final Model: LSTM + Adaptive Attention**
- **METEOR: 37.8**
- **Improvement: +0.4**
(respect to the best metric in the previous experiment)

Attention Sweep - METEOR Comparison

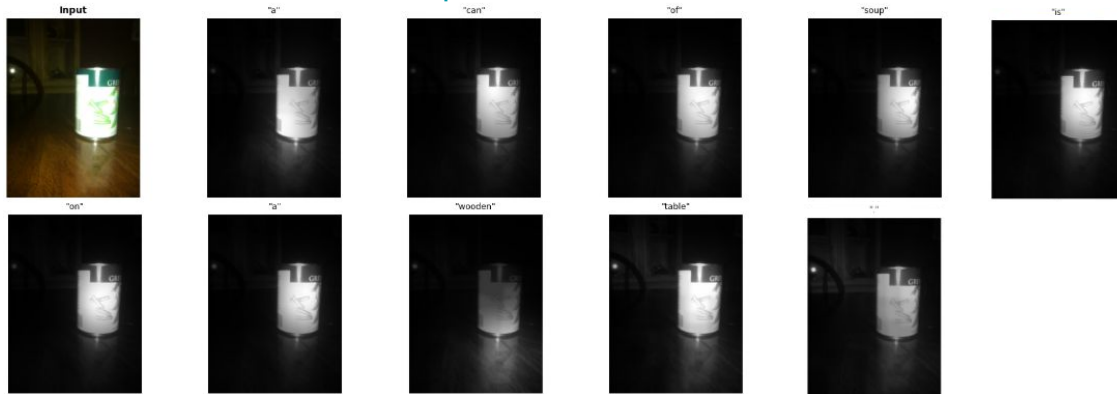


Note: In the following slides, we show qualitative examples for both **soft** and **adaptive attention**.

STEP 4: Soft Attention - Qualitative Examples

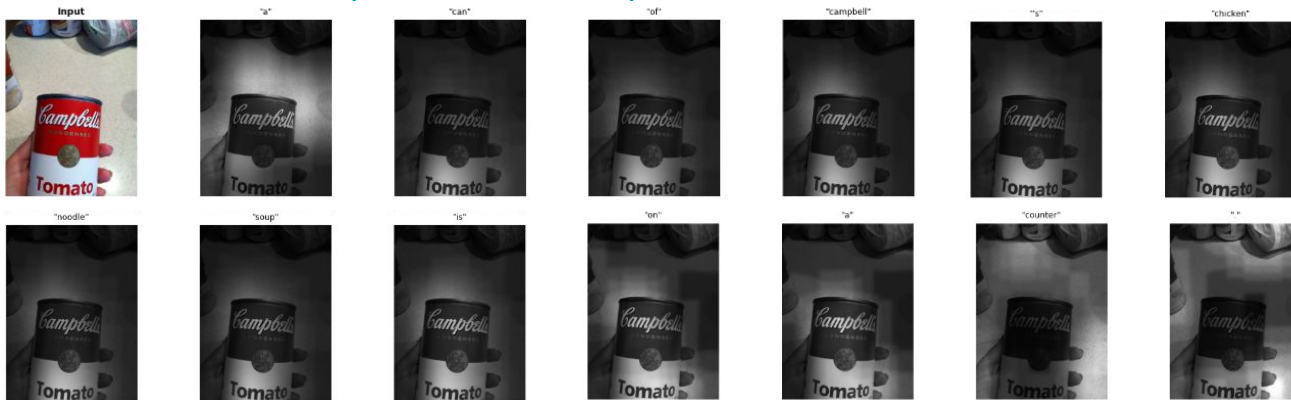
Pred: a can of soup on a wooden table

Ex 1



Pred: a can of campbell's chicken noodle soup on a counter

Ex 2

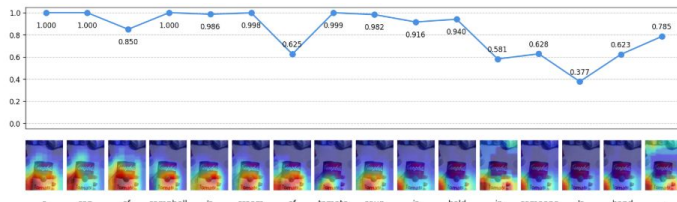


Key Observations

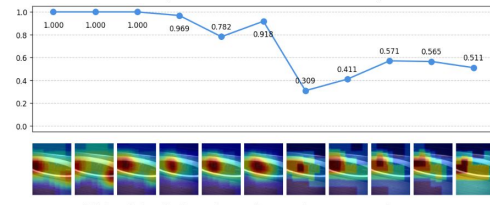
- The model is **forced to attend to the image at every timestep**, ensuring continuous visual grounding. We have based on the **visualizations of the original paper**.
- Attention maps are stable and consistently focused on the main object, with **minimal drift over time**. Some variations can be observed in **Example 1** for words such as **"wooden"** and **"table"**, where attention **shifts towards relevant contextual regions**.
- It provides **better results than the baselines but not better than adaptive attention**.

STEP 4: Adaptive Attention - Qualitative Examples

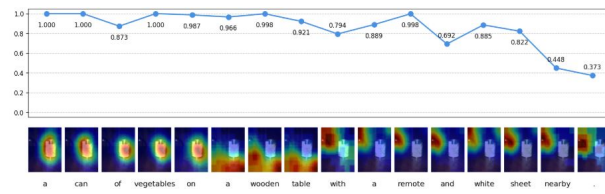
Ex 1



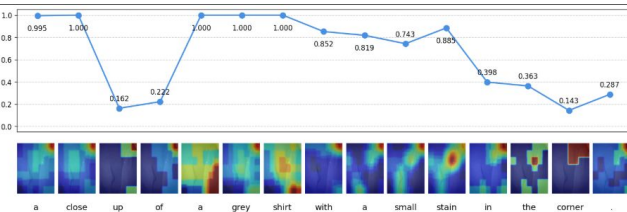
Ex 2



Ex 3



Ex 4



Key Observations

- The line chart represents $1-\beta_t$, indicating how much the model relies on **visual information vs. language context** at each timestep.
- Example 1:** Function words (e.g., "of", "s") show lower values, which is expected since **they rely more on language modeling**. In contrast, content words exhibit meaningful attention maps:
 - "Campbell": focus on the center of the can
 - "Tomato": focus on the lower region
- Examples 2 and 3:** The model behaves as expected: High values for key visual tokens. Attention maps remain semantically aligned with relevant regions.
- Example 4:** Lower and unstable values. Attention maps appear diffuse and blocky, likely due to low structure / ambiguous visual content, making it harder to extract meaningful features.

With this method we have got our best results

STEP 5: Hyperparameter search (*Beam Search*)

Motivation: Before the final hyperparameter sweep, we introduce **beam search** as it directly affects caption generation quality. Unlike greedy decoding, beam search explores multiple candidate sequences, leading to more accurate and fluent captions.

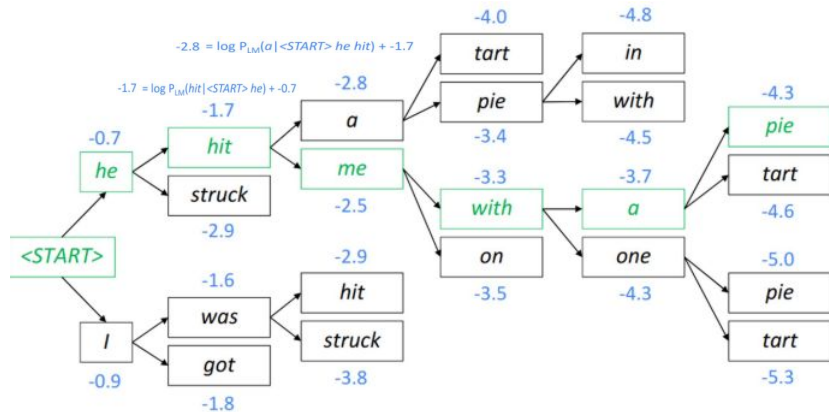
How does it Work?

- At each timestep, keep the **top K sequences (beams)**
- Expand each beam with all possible next words
- Keep the **K best sequences based on score**
- For each beam:**

$$\log P(y_{1:T}) = \sum_{t=1}^T \log P(y_t | y_{1:t-1}, x)$$

$$score = score_{prev} + \log P(next_word)$$

Beam Search with K = 2



Difference from Greedy Decoding

Differences:

- Greedy decoding** is **local optima** (selects the most probable word at each step). **Beam search** is **globally optimized** (approx) (keeps multiple candidates and selects the best overall).

Advantages of Beam Search:

- Produces more coherent and fluent captions
- Better global sequence optimization

Disadvantages of Beam Search:

- Higher computational cost (slower inference)
- Performance depends on beam size (K)

For each of the k hypothesis find top k next words and calculate scores

STEP 5: Hyperparameter search

Setup

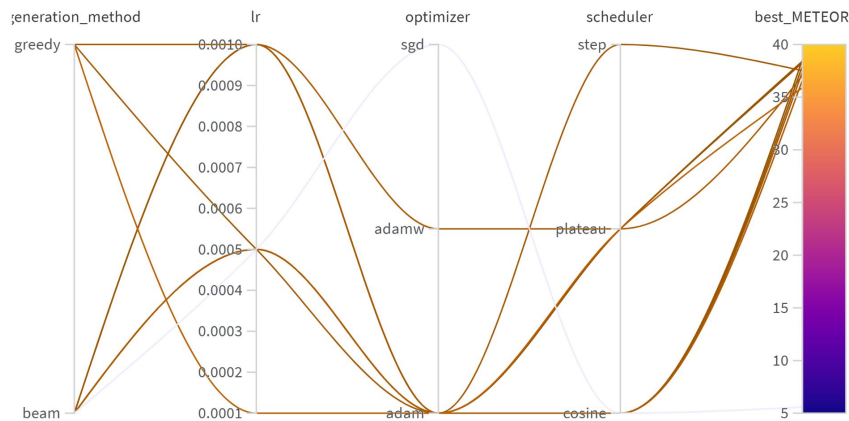
- Using the best configuration so far (**CLIP-ViT-b32 + LSTM + word + adaptive attention**), we perform a bayesian hyperparameter sweep over the following settings:
 - Optimizer:** Sgd, AdamW, Adam
 - Learning rate:** 1e-3, 5e-4, 1e-4
 - Scheduler:** Step, Plateau, Cosine
 - Decoding:** Greedy, Beam Search (k=3)

Results

- Decoding:** Beam search outperforms greedy by exploring multiple sequences, avoiding early suboptimal choices and improving coherence.
- Optimizer:** SGD underperforms due to lack of adaptive learning rates, making it slower and less effective in complex, noisy optimization like sequence generation.
- Learning rate:** Most impactful hyperparameter. Lower values (1e-4) lead to more stable training and better performance

Decision

- Final Model:**
 - Optimizer: **Adam**
 - Learning rate: **1e-4**
 - Scheduler: **Cosine**
 - Decoder: **Beam Search**
- METEOR:** 38.56
- Improvement:** +0.7



Parameter importance with respect to

Config parameter	Importance	Correlation
lr		
generation_method.value_greedy		
generation_method.value_beam		
scheduler.value_plateau		
scheduler.value_cosine		
optimizer.value_adam		
optimizer.value_adamw		
scheduler.value_step		

Final models: Generalization

Setup

- We compare the **baseline** and **final models** across two different data splits ([slide 3](#)) to assess the **generalization gap**.
- **Baseline Model:** **ResNet18** + **GRU** + **Characters**
Optimizer: **Adam** | LR: **5e-5**
Scheduler: **Plateau** | Decoding: **Greedy**
- **Final Model:** **CLIP_ViT_b32** + **LSTM** + **Word** + **Adaptive Attention**
Optimizer: **Adam** | LR: **1e-4**
Scheduler: **Cosine** | Decoding: **Beam**

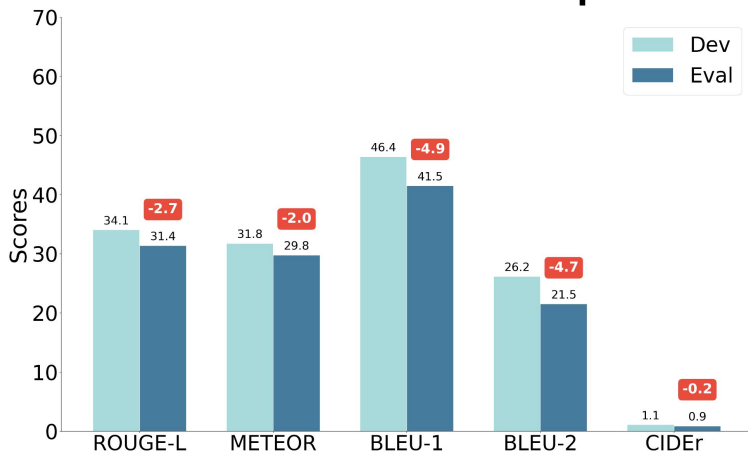
Results

- The final model shows **clear improvements** over the baseline across both data splits (Dev and Eval), with **higher scores in all the evaluated metrics**.
- Regarding generalization, the gap between Dev and Eval performance in the final model is **slightly positive** (+0.3) in METEOR, our main metric, which suggests that the model generalizes well and does not suffer from overfitting.

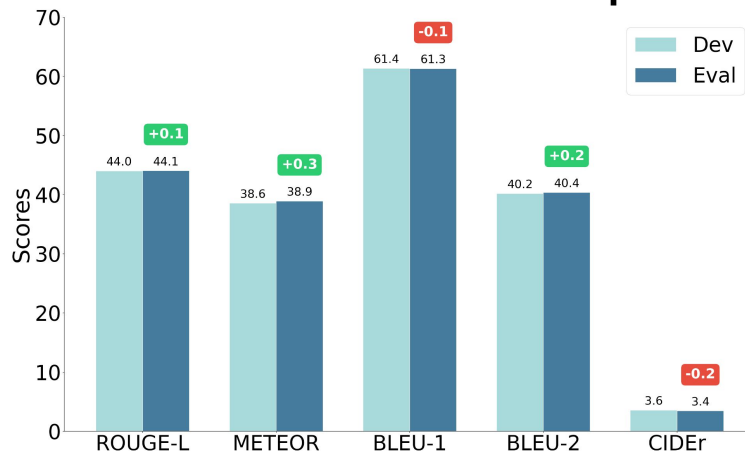
Decision

- **Improvement in Dev: +6.8**
- **Improvement in Eval: +9.1**
- **Generalization Gap for Final Model: +0.3**

Baseline: Dev vs Eval Comparison



Final Model: Dev vs Eval Comparison

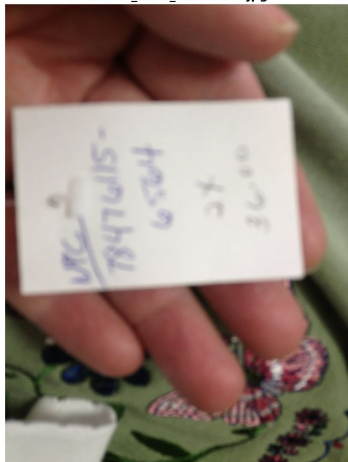


Qualitative per step model (Evolution)

Setup

- **Objective:** To visualize qualitative improvements across architectural upgrades.
- **Method:** Comparing predictions from the best epoch of each major model iteration with the ground truth captions.
- **Pipeline:** **Baseline** → +CLIP **Encoder** → +LSTM **Decoder** → +**Word-level tokens** → +**Attention** → **Best Sweep**.

VizWiz_train_00017338.jpg



Ground Truths:

1. A white label has text written in pen
2. a human hold sticker on his hand with a text closely related to phone number
3. A hand holding a piece of paper with writing from a pen on it.

Model Predictions:

Baseline (Ep 9):

A hand holding a blue box with a barcode on it.

Encoder (Ep 6):

A person holding a pink shirt with a pink shirt with a black and white label.

Decoder (Ep 9):

A person is holding a piece of paper with a picture of a person in the background.

Text-level (Ep 9):

a person holding a piece of paper with a signature on it .

Attention (Ep 7):

a person holding a small white box with a handwritten number on it

Sweep (Ep 7):

a hand is holding a white piece of paper .

VizWiz_train_00006514.jpg



Ground Truths:

1. A package of Butterball brand turkey franks hot dogs.
2. blue and beige colored bag of turkey hot dogs on a beige countertop
3. An unopened package of Butterball Turkey Franks resting on a grainy surface.
4. A package of turkey hot dogs that are lactose free.
5. A white hot dog package laying on a counter top

Model Predictions:

Baseline (Ep 9):

A package of food is on top of a table.

Encoder (Ep 6):

A package of frozen pasta sauce with a picture of the food in it.

Decoder (Ep 9):

A blue bag of frozen spinach with a blue and yellow label.

Text-level (Ep 9):

a package of chicken breast fillets is on a table .

Attention (Ep 7):

a blue and yellow package of jimmy dean hot dogs laying on a gray surface .

Sweep (Ep 7):

a blue and yellow package of butterball brand frozen food .

Conclusions

- **Final Sweep:** It consistently provides the most **logical**, detailed, and accurate descriptions of the images.
- **Early models hallucinate:** Baseline and early epoch models struggle heavily, frequently **inventing objects** not present in the scene.
- **Text recognition improves over time:** Later models successfully **detect and read text** in the wild(as the brand name).

Qualitative comparison: *baseline VS final model*

Context: We tried a qualitative comparison of the **final baseline** (best Epoch 10) and **final model** (best Epoch 7), both trained on the **full training set** and selected for their optimal METEOR scores on the validation set.

VizWiz_val_00000052



GT Captions

- A white label has a barcode on it and a Westell company logo.
- A manufacturing label with a logo and barcode.
- WHITE COLORED RECTANGULAR SHAPED PRINTED SENTENCE PLACED ON FLOOR
- a close up of a sticker of a manufacture information of a type of electronic device, that says "Westell"

Final **Baseline** Prediction

A package of food is on top of a table.

Final **Model** Prediction

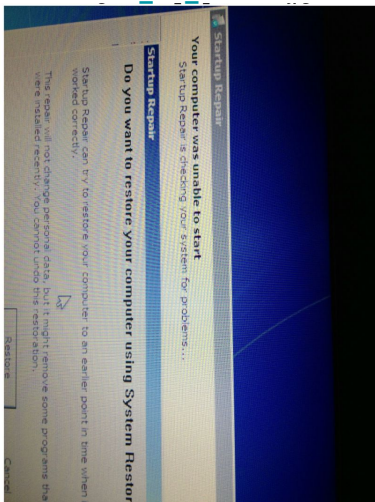
a label with a barcode and a barcode on a black surface

Conclusions

- The final model **clearly improves over the baseline**, producing a caption that better matches the ground truth by correctly identifying key elements like the **label and barcode**, whereas the baseline is semantically incorrect.
- However, the prediction still contains **redundancies** (e.g., "a barcode and a barcode") and remains too generic.

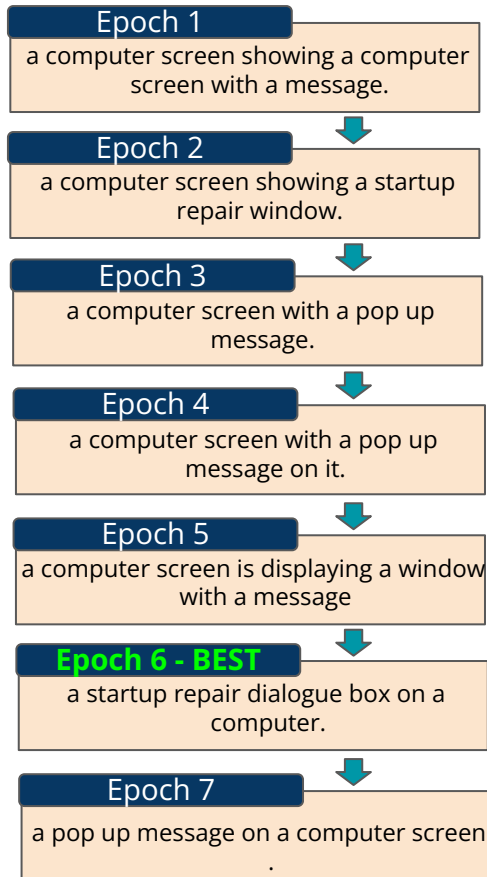
Qualitative evolution of captioning: *final model*

VizWiz_val_00000404



GT Captions

- A screen is turned on with a startup repair screen.
- Sideways view of the Startup Repair window on a computer.
- Pictured is a computer screen turned sideways in startup repair mode.
- A computer screen part blue with a pop up screen on it about repairing the computer.

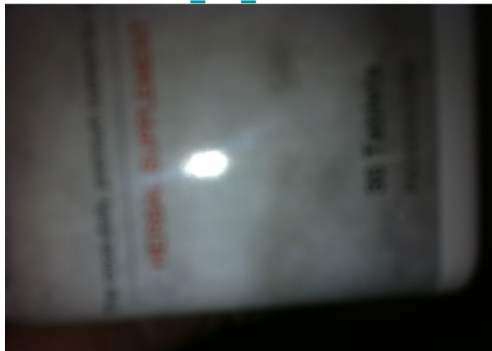


Conclusions

- **Contextual Understanding:** Comparing with the other epochs, the best one in terms of METEOR **successfully reads** the specific context of the image, instead of generating the generic description “pop up message”.
- **Learning is non-linear:** We can see how the model fluctuates during training. It starts with repetitive guesses, defaults to generic answers in the middle and finally **aligns with the ground truth** at its peak.
- **Epoch 2 vs Epoch 6:** Both capture the correct semantic meaning, but Epoch 6 **refines** from “windows” to “dialogue box”.

Qualitative final model (2 examples)

VizWiz_val_00000254



Caption - **BAD**

an advertisement for an iphone is
being held here

Caption - **GOOD**

a computer screen with a picture
of a car on it

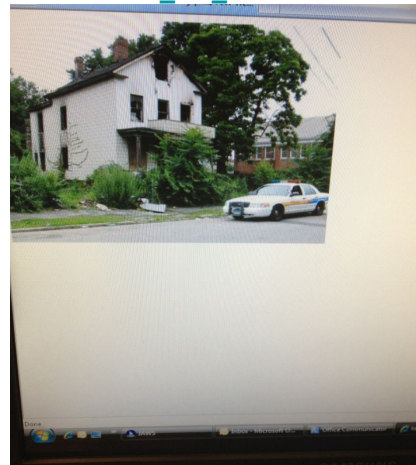
GT Captions

- A white bottle of something with writing on it.
- looks like a board in the dark with a light shining on it.

Conclusions

- The overall quality of the dataset is **limited**, which sometimes **leads the model to generate suboptimal or forced captions**, negatively affecting generalization on the evaluation set (as illustrated in the example on the left).
- For the high quality images, the model is **able to produce nice captions** like in the example on the right

VizWiz_val_00000102

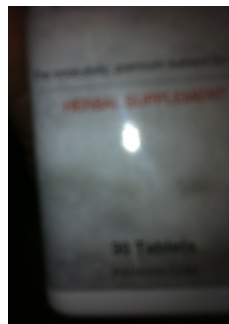
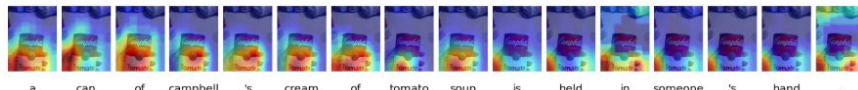
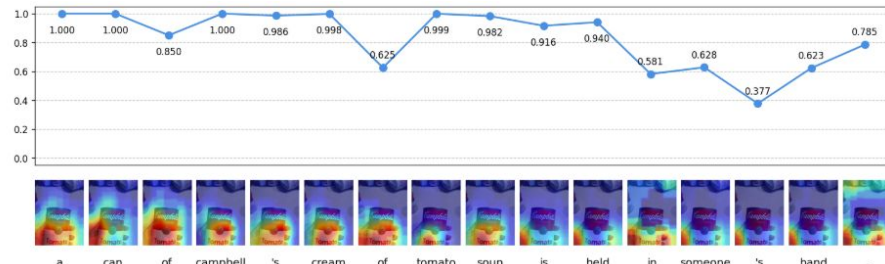
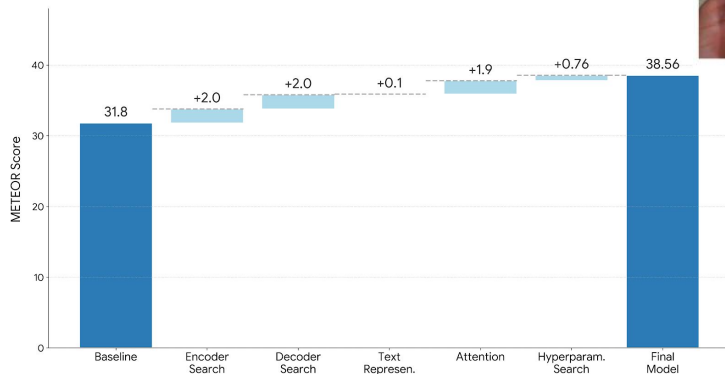


GT Captions

- A house with some police car in the monitor window
- Computer display monitor displaying old home and police vehicle.
- Computer screen display of a picture of a fire damaged house with a police car parked in front of it.
- A computer screen with a photo of an abandoned White House with an overgrown garden and a police car parked in front of it.

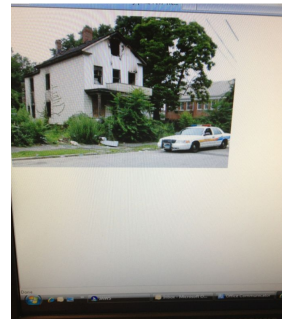
Summary

METEOR Metric Improvement Path



an advertisement for
an iphone is being
held here

a computer screen
with a picture of a
car on it



Conclusions

- **Tackling a Highly Irregular Dataset:** The VizWiz dataset presents severe challenges, featuring **images with poor lighting, extreme blur, or fingers over the lens**, paired with equally strange and noisy GT captions that heavily test the model's limits.
- **Pre-trained Semantic Alignment Wins:** The **CLIP encoder outperformed traditional CNNs** (ResNet/VGG) because its features are already aligned with text, making the decoder's job much easier.
- **Architecture vs. Attention Dynamics:** Without explicit attention, xLSTM outperforms both LSTM and GRU. However, when paired with Adaptive Attention, **LSTM ultimately surpasses xLSTM's performance**.
- **Vocabulary Granularity Matters:** **Word-level tokenization proved to be the most robust**. Subword levels struggled with sequence termination, leading to repetitive loops and severe hallucinations.
- **Knowing "When" to Look is Crucial:** **Adaptive Attention was the most effective mechanism**. By using a "visual sentinel," the model successfully learned when to ground words in the image and when to rely on its internal language model (e.g., for syntax and grammar).
- **Overall Evolution:** Through systematic upgrades (CLIP + LSTM + Word-level + Adaptive Attention), we evolved the model from outputting repetitive hallucinations to **successfully reading natural scene text** and accurately describing complex scenes.